RELATÓRIO TÉCNICO: SISTEMA DE GESTÃO ACADÊMICA

Estruturas Aninhadas, Ponteiros de Ponteiros, Alocação Dinâmica de Memória e Persistência Binária em C

Disciplina: Programação Estruturada | Curso: Engenharia da Computação
Instituição: FACENS — Faculdade de Engenharia de Sorocaba

Grupo:
Lucas Ricci Arruda Leite | RA: 263641
Daniel Martins Belculfine | RA: 263336
Thiago Samapaio Ferreira | RA: 263610

Seção 1 – Apresentação do Sistema

1.1 Problema que o Sistema Resolve

Em ambientes acadêmicos, a ausência de um controle centralizado e estruturado para o registro de estudantes e seus respectivos cursos gera dificuldades na gestão de dados cadastrais, acompanhamento de semestres, controle de carga horária concluída e persistência segura das informações. O sistema desenvolvido em linguagem C resolve essa problemática ao fornecer uma aplicação coesa para o gerenciamento de alunos, permitindo o cadastro com validação de semestres (1 a 10), vinculação de cursos, atualização seletiva de campos, exclusão lógica de registros inativos (ativo = 0) e persistência binária automática (alunos.bin). Isso evita a volatilidade dos dados entre execuções e elimina inconsistências manuais.

1.2 Usuários do Sistema

O sistema foi projetado para atender prioritariamente a esse perfil de usuário:

- Secretaria Acadêmica / Coordenadores: Responsáveis pelo cadastro inicial de alunos, matrícula em novas disciplinas e gerenciamento geral dos dados cadastrais.
- Professores / Avaliadores**:** Responsáveis pela consulta rápida de alunos ativos, filtragem de estudantes por nome de curso e acompanhamento do progresso acadêmico e carga horária concluída

1.3 Motivação da Escolha do Tema

A escolha de desenvolver um sistema de gestão acadêmica foi motivada por esse tema ser familiar para os integrantes por conta das aulas práticas de programação, além dos conceitos fundamentais poderem ser aplicados de forma prática, realista e eficiente. O domínio escolhido exige o uso de estruturas aninhadas (struct Aluno contendo a struct Curso), alocação dinâmica de memória com redimensionamento automático via realloc, tratamento de exclusão lógica para preservação de índices, além da manipulação de arquivos binários em bloco (fopen, fread, fwrite, fseek e ftell, fclose).

Seção 2 – Modelagem

2.1 Structs Criadas e Explicação de Cada Campo

1. Struct Curso (Struct Aninhada – 2 campos):

- char nome_curso[50]: Armazena o nome da formação ou curso em que o aluno está matriculado (ex: "Engenharia de Computação").
- int carga_horaria: Registra a quantidade de horas já concluídas pelo aluno no decorrer de suas disciplinas.

2. Struct Aluno (Struct Principal – 6 campos):

- int ra: Atua como o Registro Acadêmico, funcionando como identificador único do estudante no sistema.
- char nome[100]: Guarda o nome completo do estudante.
- int semestre: Indica o período letivo correspondente em que o aluno se encontra (com validação restrita de 1 a 10).
- int ativo: Flag booleana de controle para exclusão lógica (onde 1 significa aluno ativo - (ainda está na faculdade) e 0 indica registro logicamente removido (saiu da faculdade)).
- char status: Caractere que reflete a situação da matrícula (ex: 'A' para Ativo - (estudante ativo) e 'I' para Inativo – (aluno trancou o curso)).
- Curso curso_matriculado: Campo que abriga a instância aninhada da struct Curso, unificando as informações educacionais ao perfil do aluno.

2.2 Relacionamento entre as Estruturas

O relacionamento entre as estruturas ocorre por composição direta (aninhamento). Cada registro do tipo Aluno possui internamente um campo do tipo Curso chamado curso_matriculado. Dessa forma, as informações do curso pertencem exclusivamente ao escopo daquele aluno. Essa modelagem garante que, ao alocar memória dinamicamente, buscar por um aluno ou realizar operações de leitura/escrita em bloco no arquivo binário, todos os dados do curso associado sejam processados de maneira síncrona.

2.3 Por que cada informação armazenada é importante para o funcionamento do sistema?

Por estarem organizadas em blocos de memória contíguos, elas permitem que todo o estado do sistema seja gravado no arquivo binário (alunos.bin) ao encerrar a aplicação e perfeitamente reconstruído na memória RAM quando o programa é reaberto, mantendo o histórico de todos os alunos cadastrados intacto.

A importância específica de cada campo armazenado inclui:

- ra (Registro Acadêmico): É indispensável porque serve de parâmetro obrigatório nas funções de busca, atualização seletiva e remoção lógica, garantindo que o sistema localize o discente correto sem riscos de duplicidade ou ambiguidade.

- nome: Crucial para a legibilidade e identificação humana do registro nas listagens gerais e telas de feedback do sistema.

- semestre e carga_horaria: Permitem o acompanhamento quantitativo do progresso acadêmico do aluno, além de viabilizar validações de entrada (como o bloqueio de semestres inválidos fora do intervalo de 1 a 10).

- ativo e status: São fundamentais para a gestão do ciclo de vida dos dados. O uso do campo ativo = 0 (exclusão lógica) permite que o aluno seja "removido" do sistema sem a necessidade de realocar ou reordenar fisicamente o vetor dinâmico na memória RAM ou corromper os índices do arquivo binário.

- nome_curso: Permite executar a funcionalidade de listagem segmentada (filtragem por nome de curso), possibilitando que a secretaria visualize apenas os alunos matriculados em uma determinada formação.

Seção 3 – Planejamento da Implementação

3.1 Principais Funcionalidades

O sistema foi planejado e estruturado de forma modular em C para oferecer um ciclo completo de gerenciamento de dados acadêmicos:

- Cadastro de Alunos (cadastrar_aluno): Permite a inclusão de novos discentes solicitando Nome, RA, Semestre (com validação obrigatória entre 1 e 10), Nome do Curso e Horas Concluídas. A função verifica o limite da capacidade atual e aciona o redimensionamento dinâmico do vetor sempre que necessário

- Busca por RA (buscar_aluno): Localiza e exibe na tela os dados completos do aluno e do curso vinculado com base no número de Registro Acadêmico digitado, ignorando registros inativos (ativo ==0).

- Atualização Seletiva (atualizar_aluno): Permite alterar pontualmente campos específicos do registro (Nome, Semestre, Nome do Curso e Carga Horária) através de confirmações interativas no console (s/n), sem a necessidade de reescrever todo o cadastro do discente.

- Remoção Lógica (remover_aluno): Realiza a desativação do discente alterando sua flag de controle para ativo = 0, preservando a estrutura de índices do arquivo e do vetor em memória.

- Listagem Segmentada (listar_alunos): Oferece um submenu com duas opções de visualização: listagem geral de todos os discentes ativos ou filtragem de alunos matriculados em um curso específico via comparação de strings (strcmp).

- Persistência e Encerramento (salvar_e_sair e carregar_arquivo): Realiza a reconstrução do vetor dinâmico na inicialização através da leitura do arquivo binário alunos.bin e, ao encerrar o sistema, grava todos os dados atualizados em disco, mede o volume gravado em bytes (fseek/ftell) e libera a memória RAM (free).

3.2 Decisões de Projeto

Durante o desenvolvimento do software, foram adotadas as seguintes diretrizes arquiteturais:

- Exclusão Lógica em Vez de Física: Optou-se por desativar registros via flag ativo = 0 em vez de remover elementos fisicamente do vetor. Isso evita operações custosas de reordenação de memória RAM em tempo de execução e garante a integridade dos dados no arquivo binário.

- Tratamento Cuidadoso do Buffer de Entrada: Para evitar o acúmulo de sujeira no buffer do teclado (stdin) ao alternar entre leituras numéricas (scanf) e de texto (fgets), adotou-se o uso sistemático de getchar() e o tratamento da quebra de linha com strcspn(..., "\n").

- Passagem por Referência com Ponteiros: Funções que necessitam modificar o ponteiro base do vetor devido ao redimensionamento por realloc recebem o parâmetro como ponteiro para ponteiro (Aluno **cadastro), garantindo que a alteração do endereço de memória seja refletida no escopo da main.

3.3 Divisão de Responsabilidades

A construção do projeto foi organizada de maneira colaborativa entre os membros do grupo, com a seguinte divisão de tarefas:

Thiago Sampaio Ferreira: Responsável pela criação e modelagem das estruturas de dados (structs de Curso e Aluno).

Lucas Ricci Arruda Leite: Responsável pela implementação da função principal (int main) e de todas as funções estruturais relacionadas à manipulação de arquivos binários e persistência (carregar_arquivo, salvar_e_sair, com o uso de fread, fwrite, fseek, ftell e alocação/liberação de memória).

Daniel Martins Belculfine: Responsável pelo desenvolvimento das funções de regras de negócio e manipulação de registros no sistema (funções de cadastrar_aluno, buscar_aluno, atualizar_aluno, remover_aluno e listar_alunos).

Seção 4 – Registro de Uso de Inteligência Artificial

Descrição do Uso da IA

O grupo utilizou ferramentas de Inteligência Artificial Generativa (**Modo IA do Google / Gemini**) com caráter estritamente educativo e de suporte ao desenvolvimento. O principal objetivo foi entender a sintaxe, a lógica de funcionamento e os casos de uso das 6 funções essenciais para manipulação de arquivos binários em C (fopen, fclose, fread, fwrite, fseek e ftell). Além disso, a IA foi consultada para diagnosticar e depurar erros pontuais de compilação, falhas na passagem de ponteiros durante a execução do programa.

4.1 Tabela de Mapeamento do Uso da IA

Situação	O que foi solicitado à IA	Resultado Utilizado?	Modificações realizadas pelo grupo
Uso de Funções de Arquivos Binários	**Como usar e onde usar as 6 funções de arquivos binários em C (fopen, fclose, fread, fwrite, fseek e ftell).**	**Sim (Totalmente)**	**Aplicamos os conceitos diretamente nas rotinas de carregamento e salvamento estruturado do sistema.**
Correção de Erros Pontuais	**Ajuda para identificar e corrigir erros pontuais cometidos durante a escrita e compilação do código.**	**Sim (Totalmente)**	**O grupo revisou os apontamentos, ajustou a lógica no código-fonte e validou o funcionamento prático.**

4.2 Detalhamento de Correções e Alterações Realizadas

Proposta de Exclusão Física no Vetor Dinâmico

- O que a IA gerou inicialmente: Para a funcionalidade de remoção de registros, a IA sugeriu realocar o vetor dinâmico puxando todos os elementos subsequentes para trás e reduzindo a capacidade da memória RAM via realloc.

- O que foi alterado pelo grupo: Mantivemos a abordagem de exclusão lógica, apenas alterando o valor da flag ativo para 0 na struct do aluno correspondente.

- Por que a alteração foi necessária: A remoção física no vetor exigiria operações custosas de movimentação de memória e alteraria o mapeamento direto com o arquivo binário. A exclusão lógica preserva os índices e mantém a operação simples e segura.

Seção 5 – Ponteiros e Alocação Dinâmica de Memória

5.1 Onde e Por Que Ponteiros Foram Utilizados

- Onde foram utilizados: Os ponteiros aparecem em todo o código na declaração do vetor dinâmico (Aluno *cadastro), no gerenciamento do fluxo de arquivos (FILE *arq), na passagem de parâmetros para as funções de manipulação de registros (buscar_aluno, atualizar_aluno, remover_aluno, listar_alunos) e na navegação por aritmética de ponteiros ((cadastro + i)->ra).
- Por que eram necessários: Eram essenciais para evitar a cópia ineficiente de estruturas pesadas (*structs* aninhadas) na pilha (*stack*) durante a chamada das funções, permitindo o acesso direto, rápido e por referência aos dados armazenados na memória RAM.

5.2 Uso de Ponteiro para Ponteiro (Ponteiro Duplo)

- Onde e por quê: O ponteiro para ponteiro (Aluno **cadastro) foi utilizado nos parâmetros das funções cadastrar_aluno e carregar_arquivo.
- O uso de um ponteiro duplo é obrigatório nesses contextos porque as funções precisam alterar o próprio endereço base do vetor na memória RAM quando ocorre o redimensionamento por realloc. Ao passar o endereço do ponteiro (&cadastro), a alteração reflete diretamente no escopo da função main, garantindo que o ponteiro principal não aponte para um endereço inválido.

5.3 Onde Ocorreu Alocação Dinâmica e Qual Estratégia Utilizada

Onde ocorreu: A alocação dinâmica inicial ocorreu na função main(), logo na abertura do programa:

Aluno *cadastro = (Aluno *) malloc(capacidade * sizeof(Aluno));

Estratégia utilizada: Adotou-se a estratégia de dimensionamento flexível com capacidade inicial controlada (definida inicialmente para 2 elementos). Em vez de fixar um limite estático grande e desperdiçar memória RAM, o sistema aloca apenas o espaço estritamente necessário no início e expande o vetor sob demanda.

5.4 Onde Ocorreu a Realocação da Memória

Onde ocorreu: A realocação ocorreu nas funções cadastrar_aluno e carregar_arquivo.

Como funciona: Sempre que o número de elementos atinge o limite da capacidade (*total >= *capacidade), o sistema dobra o tamanho do vetor (*capacidade *= 2) e invoca a função realloc:

Aluno *temp = (Aluno *) realloc(*cadastro, (*capacidade) * sizeof(Aluno));

Essa estratégia garante que novos registros possam ser inseridos indefinidamente sem estouro de buffer. Além disso, ao término da execução na função salvar_e_sair, toda a memória alocada é devidamente devolvida ao sistema operacional por meio de free(cadastro), evitando vazamentos de memória (*memory leaks*).

Seção 6 – Arquivos Binários e Persistência

6.1 Estrutura do Arquivo Binário

Quais dados são armazenados: O sistema armazena o conjunto completo de registros dos discentes, contemplando o Registro Acadêmico (RA), o nome completo, o semestre atual, o estado de matrícula, a flag de exclusão lógica (ativo), além dos dados educacionais referentes ao curso, como o nome da formação e a quantidade de horas concluídas.
Qual estrutura é gravada: O arquivo gerado, denominado alunos.bin, armazena blocos de dados que correspondem exatamente à representação em bytes da estrutura principal Aluno, a qual contém internamente a estrutura aninhada Curso. Dessa forma, cada registro é gravado e lido de maneira estruturada em bloco contíguo, preservando o layout de memória no disco.

6.2 Explicação do Uso de fread, fwrite e fseek

- **Uso de fwrite:** É empregada na rotina de encerramento (salvar_e_sair) para gravar todo o vetor dinâmico de alunos diretamente no arquivo binário de uma só vez, transferindo o bloco contíguo de dados da memória RAM para o disco rígido.

- **Uso de fread:** É utilizada na inicialização do sistema (dentro da função carregar_arquivo) para ler de forma sequencial os registros armazenados no arquivo binário até o seu término, reconstruindo o vetor dinâmico na memória RAM antes da exibição do menu.

  Uso de fseek: É utilizada na função de salvamento em conjunto com ftell para posicionar o cursor de controle no final do arquivo (SEEK_END), viabilizando o cálculo do volume exato de bytes gravados em disco.

6.3 Persistência dos Dados

Como os dados permanecem disponíveis após o encerramento do programa?

Como a memória RAM é volátil e tem seu conteúdo apagado assim que a aplicação é fechada, o sistema resolve essa limitação gravando o estado atual do vetor dinâmico em um arquivo físico no disco (alunos.bin) no momento da saída. Quando o programa é reaberto em uma nova execução, a função de carregamento lê automaticamente esse arquivo e reconstrói todos os cadastros na memória RAM, garantindo que as informações persistam intactas entre as sessões de uso.

Seção 7 – Bateria de Testes do Sistema

Abaixo está apresentada a bateria de testes contendo 10 cenários rigorosos divididos em casos normais, limites e inválidos para validar a robustez da aplicação.

7.1 Casos Normais (Mínimo de 4)

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 1

=== CADASTRAR ALUNO ===
Nome do aluno: Lucas Ricci Arruda Leite
RA do aluno: 263641
Semestre (1 a 10): 2
Nome do curso: Engenharia da Computacao
Horas concluidas: 100

Aluno cadastrado com sucesso!
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 2

=== BUSCAR ALUNO ===
Digite o RA do aluno: 263641

----------------------------------------
ALUNO ENCONTRADO:
Nome: Lucas Ricci Arruda Leite
RA: 263641
Semestre: 2
Estado da Matricula:
Curso: Engenharia da Computacao | Horas concluidas: 100h
----------------------------------------
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 3

=== ATUALIZAR ALUNO ===
Digite o RA do aluno que deseja atualizar: 263641

Aluno encontrado: Lucas Ricci Arruda Leite
Responda com 's' para sim ou 'n' para nao.

Deseja atualizar o nome do aluno? (s/n): n
Deseja atualizar o semestre? (s/n): n
Deseja atualizar o nome do curso? (s/n): n
Deseja atualizar a quantidade de horas concluidas? (s/n): s
Nova quantidade de horas concluidas: 120

Dados atualizados com sucesso!
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 4

=== REMOVER ALUNO ===
Digite o RA do aluno que deseja remover: 263641
Aluno Lucas Ricci Arruda Leite removido com sucesso!
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 5

=== LISTAR ALUNOS ===
1 - Listar todos os alunos ativos
2 - Listar por nome de curso
Opcao: 1

--- LISTA DE TODOS OS ALUNOS ATIVOS ---
RA: 263641 | Nome: Lucas Ricci Arruda Leite | Semestre: 2 | Curso: Engenharia da Computacao | Horas Concluidas: 120h
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 5

=== LISTAR ALUNOS ===
1 - Listar todos os alunos ativos
2 - Listar por nome de curso
Opcao: 2
Digite o nome do curso para filtrar: Engenharia da Computacao

--- ALUNOS MATRICULADOS NO CURSO: Engenharia da Computacao ---
RA: 263641 | Nome: Lucas Ricci Arruda Leite | Semestre: 2 | Horas Concluidas: 120h
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 0

[Sistema] 516 bytes salvos com sucesso em 'alunos.bin'. Programa encerrado.

--------------------------------
Process exited after 240.1 seconds with return value 0
Pressione qualquer tecla para continuar. . .
```

7.2 Casos Limite (Mínimo de 2)

Caso	Entrada	Resultado Esperado
Vetor cheio	Cadastrar o 3º aluno (quando a capacidade inicial capacidade = 2 é atingida)	O sistema identifica que o vetor encheu (total >= capacidade), dispara o realloc, dobra a capacidade para 4 e realiza o cadastro do novo aluno com sucesso.
Arquivo binário inexistente	Iniciar o programa pela 1ª vez (sem o arquivo alunos.bin criado no diretório)	A função carregar_arquivo identifica que a abertura falhou (arq == NULL), encerra a tentativa de leitura sem gerar erro ou travar o sistema e inicializa o programa com o vetor dinâmico vazio.

7.3 Casos Inválidos (Mínimo de 4)

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 1

=== CADASTRAR ALUNO ===
Nome do aluno: Lucas Ricci Arruda Leite
RA do aluno: 263641
Semestre (1 a 10): 11
ERRO: Semestre invalido! Digite entre 1 e 10.
Semestre (1 a 10): -1
ERRO: Semestre invalido! Digite entre 1 e 10.
Semestre (1 a 10): 5
Nome do curso: Engenharia da Computacao
Horas concluidas: 100

Aluno cadastrado com sucesso!
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 2

=== BUSCAR ALUNO ===
Digite o RA do aluno: 263336
Aluno com RA 263336 nao encontrado.
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 7

Opcao invalida! Tente novamente.
```

```
=== SISTEMA ACADEMICO ===
1 - Cadastrar
2 - Buscar
3 - Atualizar
4 - Remover
5 - Listar
0 - Sair

Opcao: 5

=== LISTAR ALUNOS ===
1 - Listar todos os alunos ativos
2 - Listar por nome de curso
Opcao: 2
Digite o nome do curso para filtrar: Medicina

--- ALUNOS MATRICULADOS NO CURSO: Medicina ---
Nenhum registro encontrado.
```

Seção 8 – Reflexão Individual dos Integrantes

8.1 Integrante 1 – Lucas Ricci Arruda Leite

- **Qual parte do sistema você mais contribuiu?** Responsável pela implementação da função principal (int main) e de todas as funções estruturais relacionadas à manipulação de arquivos binários e persistência (carregar_arquivo, salvar_e_sair, com o uso de fread, fwrite, fseek, ftell e alocação/liberação de memória).
- **Qual conceito foi mais difícil?** O fwrite, por ser uma função nova que eu ainda não tinha visto nenhuma vez e tive que aprender sozinho.

- **O que você aprendeu sobre ponteiros?** Compreendi como a alocação dinâmica evita desperdício de memória e como operar endereços diretamente na memória RAM usando aritmética de ponteiros.
- **O que você aprendeu sobre arquivos binários?** Aprendi que arquivos binários salvam diretamente a representação da memória em bytes, tornando o processo muito mais rápido que salvar texto formatado.
- **Como a IA ajudou e quais limitações apresentou?** Me ajudou a ensinar e aplicar os principais conceitos das funções de arquivos binários.
  Não demonstrou nenhuma limitação ao me ensinar.

8.2 Integrante 2 – Daniel Martins Belculfine

- **Qual parte do sistema você mais contribuiu?** Responsável pelo desenvolvimento das funções de regras de negócio e manipulação de registros no sistema (funções de cadastrar_aluno, buscar_aluno, atualizar_aluno, remover_aluno e listar_alunos).
- **Qual conceito foi mais difícil?** Ponteiro de ponteiro, tendo que saber as partes em que eu devia usá-las ou quando usar ponteiro simples.
- **O que você aprendeu sobre ponteiros?** Aprendi a utilizar ponteiros simples para passar grandes estruturas por referência para funções de salvamento sem clonar memória.
- **O que você aprendeu sobre arquivos binários?** Entendi a função do fseek e ftell para manipular o ponteiro interno de arquivos e medir o tamanho em bytes persistido no disco.
- **Como a IA ajudou e quais limitações apresentou?** Ajudou-me, a perceber os momentos que eu devia usar os ponteiros de ponteiros, como passo o endereço de um ponteiro simples e aritméticas de ponteiros.

8.3 Integrante 3 – Thiago Sampaio Ferreira

- **Qual parte do sistema você mais contribuiu?** Criação e modelagem das estruturas de dados (structs de Curso e Aluno) e seus campos.
- **Qual conceito foi mais difícil?** Struct aninhada (Curso curso_matriculado).
- **O que você aprendeu sobre ponteiros?** Que eu posso criar uma função que receba o endereço de uma variável na main, e a partir disso, a minha função com um ponteiro consegue mudar o valor da variável, olhando para o endereço.
- **O que você aprendeu sobre arquivos binários? A** função fread é essencial para ler esses blocos de dados do disco e reconstruir o vetor dinâmico logo na inicialização do programa.
- **Como a IA ajudou e quais limitações apresentou?** Ajudou a gerar cenários de teste limite, e teve um pouco de dificuldade ao tentar ajudar a resolver o problema de realocação de memória.